

AdvR 10 – OOP Overview & S3

OOP in R

Why OOP?

Polymorphism: same function, different behaviour for different types:

```
summary(c(1, 2, 3, 4, 5))
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>         1         2         3         3         4         5
```

```
summary(factor(c("a", "b", "a")))
```

```
#> a b
#> 2 1
```

Without OOP, `summary()` would need endless if-else chains — and only the original author could add new types.

Two Paradigms

	Encapsulated	Functional
Call style	<code>object.method(arg)</code>	<code>generic(object, arg)</code>
Methods belong to	Objects/classes	Generic functions
R systems	R6, RC	S3 , S4
Other languages	Python, Java, C++	Julia

R is unusual: **functional OOP** is the default.

System	Style	Source	Use when
S3	Functional	Base R	Simple, flexible dispatch
S4	Functional	Base R	Formal, rigorous (Bioconductor)
RC	Encapsulated	Base R	Mutable objects
R6	Encapsulated	CRAN	Modern RC replacement

Priority: S3 > R6 > S4 for most tasks. S3 is used throughout base R and the tidyverse.

The sloop Package

`sloop` ("sail the seas of OOP") helps inspect OOP objects:

```
otype(1:10)          # base object
```

```
#> [1] "base"
```

```
otype(mtcars)        # S3 object
```

```
#> [1] "S3"
```

```
ftype(print)         # S3 generic
```

```
#> [1] "S3"      "generic"
```

```
ftype(str)           # S3 generic
```

```
#> [1] "S3"      "generic"
```

```
ftype(t.test)        # S3 generic (not a method!)
```

```
#> [1] "S3"      "generic"
```

Base Types

Base vs. OO Objects

OO objects have a **class attribute**; base objects don't:

```
is.object(1:10)
```

```
#> [1] FALSE
```

```
attr(1:10, "class")
```

```
#> NULL
```

```
is.object(mtcars)
```

```
#> [1] TRUE
```

```
attr(mtcars, "class")
```

```
#> [1] "data.frame"
```

Every object has a **base type** (via `typeof()`), but only OO objects have a class.

Common Base Types

```
typeof(1L)      # integer
```

```
#> [1] "integer"
```

```
typeof(1.5)     # double
```

```
#> [1] "double"
```

```
typeof(TRUE)    # logical
```

```
#> [1] "logical"
```

```
typeof("hello") # character
```

```
#> [1] "character"
```

```
typeof(mean)    # closure (function)
```

```
#> [1] "closure"
```

25 base types total. Key groups: vectors, functions, environments, language components.

Caution: “numeric” means three different things in R — alias for `double`, shorthand for integer-or-double, or “behaves like a number” (`is.numeric()`).

S3 Basics

What Is an S3 Object?

A base type + a class attribute:

```
f <- factor(c("a", "b", "c"))
typeof(f)
```

```
#> [1] "integer"
```

```
attributes(f)
```

```
#> $levels
```

```
#> [1] "a" "b" "c"
```

```
#>
```

```
#> ... [output truncated]
```

Strip the class with `unclass()`:

```
unclass(f)
```

```
#> [1] 1 2 3
```

```
#> attr(,"levels")
```

```
#> ... [output truncated]
```

A **generic** dispatches to a **method** based on the object's class:

```
ftype(print)    # "generic" = dispatches
```

```
#> [1] "S3"      "generic"
```

```
s3_dispatch(print(f))
```

```
#> => print.factor
```

```
#> * print.default
```

- `=>` = method called; `*` = method exists but not called
- Methods are named `generic.class()` (e.g., `print.factor()`)

Never call `print.factor()` directly — always use the generic.

The generic.class() Trap

The . in function names is ambiguous:

```
fftype(t.test)      # generic, not a method!
```

```
#> [1] "S3"      "generic"
```

```
fftype(t.data.frame) # method for t() on data frames
```

```
#> [1] "S3"      "method"
```

`t.test()` is a **generic** function for t-tests. `t.data.frame()` is a **method** for the `t()` generic.

When unsure, check with `fftype()`.

- ❶ Describe the difference in behaviour:

```
set.seed(1014)
some_days <- as.Date("2017-01-31") + sample(10, 5)
mean(some_days)
mean(unclass(some_days))
```

- ❷ What class, base type, and attributes does this return?

```
x <- table(rpois(100, 5))
```

Solutions (Basics)

- ❶ `mean(some_days)` dispatches to `mean.Date()`, returning a `Date`. `mean(unclass(some_days))` strips the class, so `mean.default()` returns a raw numeric (days since 1970-01-01).

```
set.seed(1014)
some_days <- as.Date("2017-01-31") + sample(10, 5)
c(mean(some_days), mean(unclass(some_days)))
```

```
#> [1] "2017-02-06" "2017-02-06"
```

- ❷ Class table, base type integer, with `dimnames` attribute:

```
x <- table(rpois(100, 5))
c(class(x), typeof(x))
```

```
#> [1] "table"      "integer"
```

```
names(attributes(x))
```

```
#> [1] "dim"          "dimnames" "class"
```

Creating S3 Classes

Three Functions for Every Class

- ① **Constructor** `new_myclass()` — creates objects with correct structure
- ② **Validator** `validate_myclass()` — checks values are valid
- ③ **Helper** `myclass()` — user-friendly creation

Constructor Example: Date

A Date is a double with class "Date":

```
new_Date <- function(x = double()) {  
  stopifnot(is.double(x))  
  structure(x, class = "Date")  
}  
new_Date(c(-1, 0, 1))
```

```
#> [1] "1969-12-31" "1970-01-01" "1970-01-02"
```

A difftime also has a units attribute:

```
new_difftime <- function(x = double(), units = "secs") {  
  stopifnot(is.double(x))  
  units <- match.arg(units,  
    c("secs", "mins", "hours", "days", "weeks"))  
  structure(x, class = "difftime", units = units)  
}  
new_difftime(52, "weeks")
```

```
#> Time difference of 52 weeks
```

Validators

Constructors check **types**; validators check **values**:

```
new_factor <- function(x = integer(), levels = character()) {  
  stopifnot(is.integer(x)); stopifnot(is.character(levels))  
  structure(x, levels = levels, class = "factor")  
}  
  
validate_factor <- function(x) {  
  values <- unclass(x); levels <- attr(x, "levels")  
  if (!all(!is.na(values) & values > 0))  
    stop("All values must be > 0", call. = FALSE)  
  if (length(levels) < max(values))  
    stop("Not enough levels", call. = FALSE)  
  x  
}  
  
validate_factor(new_factor(1:5, "a"))
```

```
#> Error: Not enough levels
```

Helpers provide a **user-friendly** interface:

```
factor2 <- function(x = character(), levels = unique(x)) {  
  ind <- match(x, levels)  
  validate_factor(new_factor(ind, levels))  
}  
factor2(c("a", "a", "b"))
```

```
#> [1] a a b
```

```
#> Levels: a b
```

Pattern: helper → coerce inputs → call constructor → call validator.

Generics and Methods

Creating a Generic

A generic calls `UseMethod()` to dispatch:

```
my_generic <- function(x, ...) {  
  UseMethod("my_generic")  
}  
  
my_generic.numeric <- function(x, ...) {  
  paste("Numeric:", sum(x))  
}  
  
my_generic.character <- function(x, ...) {  
  paste("Character:", paste(x, collapse = ", "))  
}  
  
my_generic(1:3)
```

```
#> [1] "Numeric: 6"
```

```
my_generic(c("a", "b"))
```

```
#> [1] "Character: a, b"
```

`UseMethod()` builds `paste0("generic", ".", class(x))` and searches:

```
s3_dispatch(print(Sys.Date()))
```

```
#> => print.Date  
#> * print.default
```

The **default** pseudo-class provides a fallback:

```
s3_dispatch(mean(1:5))
```

```
#>    mean.integer  
#>    mean.numeric  
#> => mean.default  
#> ... [output truncated]
```

Finding Methods

```
s3_methods_generic("mean")
```

```
#> # A tibble: 7 x 4  
#>   generic class      visible source  
#>   <chr>   <chr>    <lgl>   <chr>  
#> 1 mean    Date      TRUE    base  
#> 2 mean    default  TRUE    base  
#> ... [output truncated]
```

```
s3_methods_class("ordered")
```

```
#> # A tibble: 4 x 4  
#>   generic      class      visible source  
#>   <chr>       <chr>    <lgl>   <chr>  
#> 1 as.data.frame ordered TRUE    base  
#> 2 Ops         ordered TRUE    base  
#> ... [output truncated]
```

- 1 Read the source of `t()` and `t.test()`. Confirm that `t.test()` is a generic, not a method for `t()`.
- 2 What generics does the `table` class have methods for?

Solutions (Generics)

- ❶ `t()` is a generic that transposes. `t.test()` is a **separate generic** for t-tests (not the `test` method of `t()`):

```
fztype(t)      # generic
```

```
#> [1] "S3"      "generic"
```

```
fztype(t.test) # generic (not t.method!)
```

```
#> [1] "S3"      "generic"
```

❷

```
s3_methods_class("table")
```

```
#> # A tibble: 10 x 4
```

```
#>   generic      class visible source
```

```
#>   <chr>      <chr> <lgl>  <chr>
```

```
#> 1 [         table TRUE   base
```

```
#> 2 aperm     table TRUE   base
```

```
#> ... [output truncated]
```

Object Styles and Inheritance

Four Object Styles

Style	Length means	Example
Vector	# observations	Date, factor
Record	# components	POSIXlt (list of 11)
Data frame	# columns	data.frame, tibble
Scalar	1 thing	lm (list of 12)

```
mod <- lm(mpg ~ wt, data = mtcars)
length(mod)  # 12 components, but 1 model
```

```
#> [1] 12
```

A class can be a **character vector** — R searches methods left to right:

```
class(ordered("x"))
```

```
#> [1] "ordered" "factor"
```

```
s3_dispatch(print(ordered("x")))
```

```
#>    print.ordered
```

```
#> => print.factor
```

```
#> * print.default
```

- ordered is a **subclass** of factor
- factor is a **superclass** of ordered

NextMethod()

`NextMethod()` delegates to the next method in the class hierarchy:

```
new_secret <- function(x = double()) {  
  stopifnot(is.double(x))  
  structure(x, class = "secret")  
}  
  
print.secret <- function(x, ...) {  
  print(strrep("x", nchar(x)))  
  invisible(x)  
}  
  
`[.secret` <- function(x, i) {  
  new_secret(NextMethod())  
}  
  
x <- new_secret(c(15, 1, 456))  
x[1]
```

```
#> [1] "xx"
```

```
s3_dispatch(x[1])
```

```
#> => [.secret  
#>    [.default  
#> -> [ (internal)
```

```
#>    [output truncated]
```

Four **group generics** override behaviour for entire families:

Group	Functions
Math	<code>abs()</code> , <code>sqrt()</code> , <code>log()</code> , <code>cos()</code> , ...
Ops	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>==</code> , <code><</code> , <code>&</code> , <code>\ </code> , ...
Summary	<code>sum()</code> , <code>min()</code> , <code>max()</code> , <code>range()</code> , ...
Complex	<code>Re()</code> , <code>Im()</code> , <code>Mod()</code> , ...

```
s3_dispatch(abs(as.difftime(10, units = "mins")))
```

```
#>    abs.difftime
#>    abs.default
#> => Math.difftime
#> ... [output truncated]
```

Ops group uses **double dispatch** (dispatches on both arguments).

Exercises (Inheritance)

❶ What does this code return? Why?

```
generic2 <- function(x) UseMethod("generic2")
generic2.a1 <- function(x) "a1"
generic2.a2 <- function(x) "a2"
generic2.b <- function(x) {
  class(x) <- "a1"
  NextMethod()
}
generic2(structure(list(), class = c("b", "a2")))
```

Solutions (Inheritance)

- ① Returns "a2". Even though `generic2.b` changes the class to "a1", `NextMethod()` ignores the modified class. It uses the **original** class vector `c("b", "a2")` and dispatches to the next in line: `generic2.a2`.

```
generic2 <- function(x) UseMethod("generic2")
generic2.a1 <- function(x) "a1"
generic2.a2 <- function(x) "a2"
generic2.b <- function(x) {
  class(x) <- "a1"
  NextMethod()
}
generic2(structure(list(), class = c("b", "a2")))
```

```
#> [1] "a2"
```